

Teoría de la Computabilidad

Módulo 14:
Funciones recursivas parciales:
extendiendo las funciones recursivas primitivas
con el operador de minimización

Departamento de Cs. e Ing. de la Computación
 Universidad Nacional del Sur
 Bahía Blanca, Argentina

1

Repaso

Previamente presentamos las nociones:

- Función Recursiva Primitiva.
 - Predicado Recursivo Primitivo
- Demostrado a través de la función característica asociada al predicado.
- Función Definida por Casos

Existen predicados más complejos. Estos se definen utilizando cuantificación.

2

Cuantificación $\exists$ / $\forall$ acotada

Def.: Sea $n \geq 0$. Si P es un predicado $(n+1)$ -ario, entonces su **cuantificación existencial acotada** es el predicado Q , $(n+1)$ -ario, tal que para cada $(x_1, x_2, \dots, x_n) \in \text{Nat}^n$, y $m \in \text{Nat}$, $Q((x_1, x_2, \dots, x_n), m)$ es verdadero sssi $\exists i, 0 \leq i \leq m$, tal que $P((x_1, x_2, \dots, x_n), i)$ es verdadero.

Abreviadamente:

$$Q((x_1, \dots, x_n), m) \text{ sssi } (\exists i)_{\leq m} [P((x_1, x_2, \dots, x_n), i)]$$

Análogamente, la cuantificación universal acotada de un predicado R se define como

$$R((x_1, \dots, x_n), m) \text{ sssi } (\forall i)_{\leq m} [P((x_1, x_2, \dots, x_n), i)]$$

3

Ejemplos

Veamos ejemplo de cómo podría definir Q y P :

$$Q((x_1, \dots, x_n), m) \text{ sssi } (\exists i)_{\leq m} [P((x_1, x_2, \dots, x_n), i)]$$

Ejemplo: supongamos $\text{menor}(x, y)$ es verdadero sssi “ x es menor que y ”, y falso en caso contrario

$$\text{existemenor}(x_1, x_2) \text{ sssi } (\exists i)_{\leq x_2} [\text{menor}(x_1, i)]$$

$$\text{existemenor}(3, 2) \text{ sssi } (\exists i)_{\leq 2} [\text{menor}(3, i)]$$

Luego $\text{existemenor}(3, 2)$ es falso (pues **no existe** ningún i que cumpla la definición).

4

Ejemplos

Veamos ejemplo de cómo podría definir R y P :

$$R((x_1, \dots, x_n), m) \text{ sssi } (\forall i)_{\leq m} [P((x_1, x_2, \dots, x_n), i)]$$

Ejemplo: supongamos $\text{moig}(x, y)$ es verdadero sssi “ x es menor o igual que y ”, y falso en caso contrario

$$\text{siempremoig}(x_1, x_2) \text{ sssi } (\forall i)_{\leq x_2} [\text{moig}(x_1, i)]$$

$$\text{Ej. siempremoig}(0, 2) \text{ sssi } (\forall i)_{\leq 2} [\text{moig}(0, i)]$$

Luego $\text{siempremoig}(0, 2)$ es verdadero (pues **para todos** los valores de i , entre 0 y 2, se satisface que $\text{moig}(0, i)$ es verdadero).

5

Lema sobre Cuantificación Acotada

Lema: Si P es un predicado recursivo primitivo, entonces los predicados correspondientes a

su cuantificación existencial acotada

$$(\exists i)_{\leq m} [P((x_1, x_2, \dots, x_n), i)]$$

y su cuantificación universal acotada

$$(\forall i)_{\leq m} [P((x_1, x_2, \dots, x_n), i)]$$

son también predicados recursivos primitivos.

6

Lema sobre Cuantificación Acotada**Esquema demostración [1]:**

Sea $(\exists i)_{\leq m} [P((x_1, x_2, \dots, x_n), i)]$ y $f_P((x_1, x_2, \dots, x_n), i)$ la fc. característica de $P((x_1, x_2, \dots, x_n), i)$. Es sencillo comprobar que la cuant. existencial acotada es representable mediante los predicados:

$$[\Sigma_{i=0}^m f_P((x_1, x_2, \dots, x_n), i)] \neq 0,$$

$$[\Sigma_{i=0}^m f_P((x_1, x_2, \dots, x_n), i)] > 0,$$

$$\text{o bien } \text{sg}[\Sigma_{i=0}^m f_P((x_1, x_2, \dots, x_n), i)] = 1.$$

7

Lema sobre Cuantificación Acotada**Esquema dem [2]:**

Ejercicio:
completar
detalles dem.

Todos los casos anteriores ellos pueden ser reescritos utilizando definiciones y ejemplos ya considerados. Se satisfacen así los requerimientos de la def. de predicado recursivo primitivo.

Análogam. es el caso para

$$(\forall i)_{\leq m} [P((x_1, x_2, \dots, x_n), i)]$$

que puede representarse p.ej. por $\prod_{i=0}^m f_P((x_1, x_2, \dots, x_n), i) = 1$.

8

Ejemplo con Cuantificación Acotada

Sea la función característica

$$f_{\text{divisor}}: \text{Nat} \times \text{Nat} \rightarrow \{0, 1\},$$

donde $f_{\text{divisor}}(a, b) = a|b$ es interpretable como “a es divisor de b” o “b es divisible por a”. Usamos la convención: 1= respuesta afirmativa, 0=resp. negativa.

Esta función, si la pudieramos definir como FRP, sería la función característica del predicado recursivo primitivo **esdivisor(a, b)** (que sería “verdadero” si a es divisor de b y falso en caso contrario).

9

Ejemplo con Cuantificación Acotada

Luego podemos utilizar la interpretación de la función como un algoritmo especificado informalmente como sigue:

$$a|b \text{ sssi } (\exists i)_{\leq b} (i * a = b)$$

$$\text{esdivisor}(a, b) = (\exists i)_{\leq b} (i * a = b)$$

$$\text{Ejemplo: } 4|60 \text{ sssi } (\exists i)_{\leq 60} (i * 4 = 60)$$

10

Ejemplo con Cuantificación Acotada

Suponiendo que mediante f_{igual} denotamos a la fc. característica del pred. igual, podemos refinar lo anterior a través de la expresión:

$$f_{\text{divisor}}(x, y) = f_{\text{igual}}(\text{sg}[\Sigma_{i=0}^y f_{\text{igual}}(x * i, y)], 1)$$

Especificación final: queda como ejercicio. Notemos que el test por igualdad con 1 que se realiza al final del ej. no es estrictamente necesario (fue incluido para adecuar la solución al esquema general antes sugerido).

11

Anidamiento de Cuantificadores

En matemáticas pueden construirse expresiones que requieran cuantificadores anidados:

$$\exists x \exists y (suc(x) = y)$$

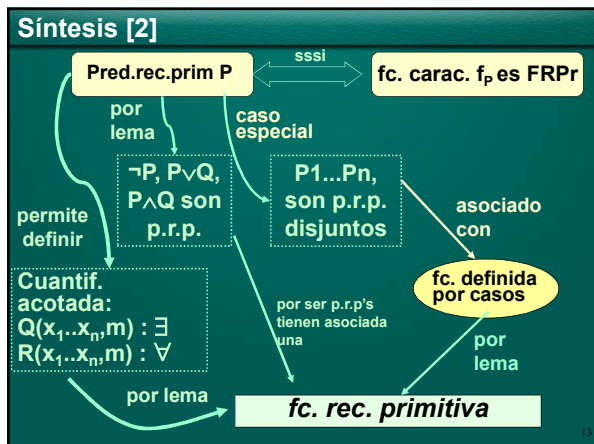
$$\exists x \forall i (x < y)$$

$$\forall x \exists y (x < y)$$

$$\forall x \forall y (x < y \text{ ó } x = y \text{ ó } x > y)$$

Los cuantificadores acotados que hemos visto pueden anidarse análogamente, para obtener expresiones más sofisticadas.

12



Funciones no Recursivas Primitivas

- Una pregunta interesante: Todas las funciones Turing-computables ¿son fcs. recursivas primitivas?
- **Respuesta: no.** Se pueden construir funciones *totales* que son Turing-computables, pero no son recursivas primitivas.
- Ejemplo: la *función de Ackerman*. Esta fc. fue propuesta por Ackerman en 1928 como respuesta a un problema planteado por Hilbert en 1925.

Función de Ackerman

$\text{ack}(a_1, a_2) : \text{Nat} \times \text{Nat} \rightarrow \text{Nat}$

Si $a_1 = 0$
 entonces
 retornar $a_2 + 1$
 si no
 si $a_2 = 0$
 entonces
 retornar $\text{ack}(a_1 - 1, 1)$
 si no
 retornar $\text{ack}(a_1 - 1, \text{ack}(a_1, a_2 - 1))$

Dos
 caracterizaciones
 alternativas de la
 función de
 Ackermann

$\text{ack}(0, a_2) = a_2 + 1$
 $\text{ack}(a_1 + 1, 0) = \text{ack}(a_1, 1)$
 $\text{ack}(a_1 + 1, a_2 + 1) = \text{ack}(a_1, \text{ack}(a_1 + 1, a_2))$

Función de Ackerman

$\text{ack}(0, a_2) = a_2 + 1$
 $\text{ack}(a_1 + 1, 0) = \text{ack}(a_1, 1)$
 $\text{ack}(a_1 + 1, a_2 + 1) = \text{ack}(a_1, \text{ack}(a_1 + 1, a_2))$

$\text{ack}(0, 3) = 4$
 $\text{ack}(3, 0) = \text{ack}(2, 1)$ y para calcular $\text{ack}(2, 1)$:

$\text{ack}(2, 1) = \text{ack}(1, \text{ack}(2, 0))$
 $\text{ack}(2, 0) = \text{ack}(1, 1) = 3$
 $\text{ack}(1, 1) = \text{ack}(0, \text{ack}(1, 0)) = \text{ack}(0, 2) = 3$
 $\text{ack}(1, 0) = \text{ack}(0, 1) = 2$
 $\text{ack}(1, 3) = \text{ack}(0, \text{ack}(1, 2)) = \text{ack}(0, 4) = 5$
 $\text{ack}(1, 2) = \text{ack}(0, \text{ack}(1, 1)) = \text{ack}(0, 3) = 4$

Algo sobre Wilhelm Ackermann

Nació el 29 de Marzo de 1896 (Schönebeck, Alemania), murió el 24 de Diciembre de 1962 (Lüdenscheid, Alemania).

Ackermann se doctoró en 1925 con una tesis dirigida por Hilbert. De 1927 a 1961 fue profesor en un colegio alemán.

En 1928 se le ocurrió una fc. $A(x, y, z)$ recursiva que no es primitiva. Esta fc. $A(x, y, z)$ fue simplificada a una fc. de 2 variables $P(x, y)$ por la matemática húngara Rosza Peter. Esta última versión es la que llegó a nuestros días.

Función de Ackerman

$\text{ack}(0, a_2) = a_2 + 1$
 $\text{ack}(a_1 + 1, 0) = \text{ack}(a_1, 1)$
 $\text{ack}(a_1 + 1, a_2 + 1) = \text{ack}(a_1, \text{ack}(a_1 + 1, a_2))$

Ackerman difiere de las fcs. primitivas estudiadas, pues se basa en el concepto de recursividad anidada (=se define el nuevo valor de ack usando valores de ack para argumentos que son valores de la fc. en sí mismos).

Puede demostrarse que la función de Ackermann no es recursiva primitiva (no se verá la demostración). Este hecho es relevante para el siguiente teorema.

Teorema: Fc.Rec.Prim $\rightarrow$ Fc.Total

Teo.: Toda función recursiva primitiva es una función total.

La recíproca del Teorema no es cierta (justificada por la existencia de la fc. de Ackerman).

Demostración:

Por inducción sobre la longitud de la derivación más corta de cada fc. rec. primitiva f .

Paso Base: Si f tiene una derivación de longitud uno es porque f es una función base (cero, suc, o Π^n). En cada caso, se trata de funciones totales.

19

Teorema: Fc.Rec.Prim $\rightarrow$ Fc.Total

Hipotesis Inductiva: Supongamos que toda fc.rec.prim. definible a través de una derivación formal de longitud $\leq k$ es total.

Paso Inductivo: Sea f una fc. que requiere una derivación mínima de $k+1$ elementos.

Si f es obtenida desde las fc. recursivas primitivas $g_1, g_2, \dots, g_n$ y h por **composición**, ent. c/u de estas fc. deben tener derivaciones de longitud $\leq k$. La hip. inductiva ent. asegura que $g_1, g_2, \dots, g_n$ y h son totales, de donde se deduce que f es total.

Análogamente, si f es obtenida desde fc. rec.prim. g y h por **recursión primitiva**, la hipótesis de inducción implica que g y h son totales. Luego, por def. de recursión primitiva, f es total.

20

Funciones Recursivas

- Se denominan **Funciones Recursivas** a aquellas que son funciones totales. Por ejemplo, la función de Ackerman es una función recursiva.
- La mayoría de las funciones usadas habitualmente son recursivas primitivas
- Tanto es así, que por mucho tiempo se creyó que todas las funciones totales se podían representar con funciones recursivas primitivas. El resultado de Ackerman demostró lo contrario.

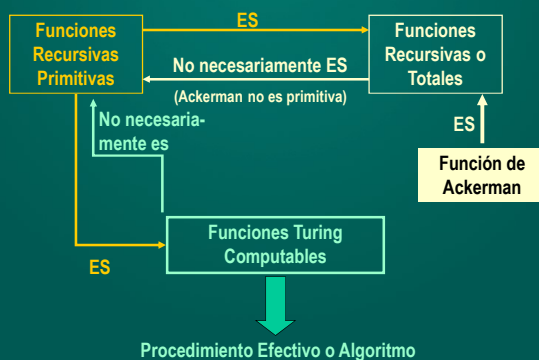
21

Funciones Recursivas Parciales

- Las funciones recursivas primitivas no alcanzan para caracterizar las funciones Turing-computables.
- No todos los algoritmos producen una salida para todas las entradas (es decir, existen algoritmos que no se detienen).
- Esto sugiere que para incluir las funciones computadas por todos los algoritmos es necesario incluir las funciones parciales.
- Consideraremos entonces una nueva forma de construir funciones.

22

Síntesis



23

Propuesta



24

Funciones Recursivas Parciales

Def.: Sea $n \geq 0$, g una función $(n+1)$ -aria, y $\mu_y(g(x_1..x_n, y)=0)$ el mínimo natural y tal que $g(x_1..x_n, y)$ es igual a 0. La **minimización no acotada** de g es una función n -aria f , tal que para cualquier $(x_1..x_n) \in \text{Nat}^n$ se tiene:

$f(x_1..x_n) =_{\text{def}} \mu_y(g(x_1..x_n, y)=0)$ si tal y existe
Si no existe tal y , la fc. f queda indefinida.

25

Funciones Recursivas Parciales

Def: Una fc. $f: \text{Nat}^n \rightarrow \text{Nat}$ es **recursiva parcial** si

- Es una función base o inicial
 - Es una fc. construida a partir de las funciones recursivas iniciales usando **composición**, **recursión** o **minimización no acotada**.
- Ninguna otra función es recursiva parcial.

Notemos que se agrega **minimización no acotada** (nuevo constructor, los otros ya estaban disponibles para funciones recursivas primitivas).

26

Func. Rec. Parciales: Ejemplo

Sea $f: \text{Nat} \rightarrow \text{Nat}$ definida como $f(a) =_{\text{def}} \lfloor a/2 \rfloor$ denotando la parte entera de $a/2$. Algunos valores de $f(X)$ serían:

$f(0) = \lfloor 0/2 \rfloor = 0$, $f(1) = \lfloor 1/2 \rfloor = 0$, $f(2) = \lfloor 2/2 \rfloor = 1$
 $f(3) = \lfloor 3/2 \rfloor = 1$, $f(4) = \lfloor 4/2 \rfloor = 2$, $f(5) = \lfloor 5/2 \rfloor = 2$

Si bien en este caso f es una fc. total recursiva primitiva (ejercicio) supongamos querer mostrar que **$b = \lfloor a/2 \rfloor$** puede expresarse usando minimización no acotada.

Idea: reescribimos la función como $y = \lfloor x/2 \rfloor$, a partir de donde podemos expresar la desigualdad:

$$y \leq x/2 < y+1$$

27

Func. Rec. Parciales: Ejemplo

$$y \leq x/2 < y+1$$

Buscamos llegar a una expresión de la forma $g(x, y)=0$, usando siempre operaciones matemáticas que sean "recursivas primitivas". Luego podríamos reescribir lo anterior como sigue:

$$2y \leq x < 2y+2$$

Sabemos que x es natural, y por lo tanto mayor o igual a cero. Entonces, en particular, la parte derecha de la desigualdad es reexpresable como

$$x \leq 2y+1.$$

En notación informal de fc.rec.prim, se tiene: $(x + 2y) + 1 = 0$

28

Sabemos que x es natural, y por lo tanto mayor o igual a cero. Entonces, en particular, la parte derecha de la desigualdad es reexpresable como

$$x \leq 2y+1.$$

Podríamos aplicar diferencia primitiva en ambos lados de la desigualdad, y resulta:

$$x + 2y \leq (2y+1) + 2y$$

$$x + 2y + 1 \leq (2y+1) + 2y + 1$$

Pero la expresión a la derecha del $\leq$ equivale a 0. Luego resulta que: $x + 2y + 1 \leq 0$

Como la expresión se calcula SOLAMENTE con números naturales, puedo asegurar que buscar satisfacer la desigualdad anterior es equivalente a:

$$x + 2y + 1 = 0$$

29

Func. Rec. Parciales: Ejemplo

$$x + 2y + 1 = 0$$

Notemos que hay muchos pares de naturales (x, y) que satisfacen esta igualdad. Pero... el valor " x " es el valor que me dan de la función (ej. $f(3)$) para calcular el " y ". Luego, una vez que me dan " x ", podría buscar cuál es el valor de " y " para satisfacer la igualdad.

Y eso es precisamente lo que hace minimización (comenzando con $y=0$, podemos probar distintos valores de " y " hasta encontrar el que satisfaga la igualdad).

$F(x)$ es ent. el mínimo y tal que $(x + 2y) + 1 = 0$

Verifiquemos si los resultados de la fc. se corresponden con los de $f(x) = \mu_y ((x + 2y) + 1 = 0)$.

$f(0) = \lfloor 0/2 \rfloor = 0$, $f(1) = \lfloor 1/2 \rfloor = 0$, $f(2) = \lfloor 2/2 \rfloor = 1$
 $f(3) = \lfloor 3/2 \rfloor = 1$, $f(4) = \lfloor 4/2 \rfloor = 2$, $f(5) = \lfloor 5/2 \rfloor = 2$

30

Func. Rec. Parciales: Ejemplo

x	y	$(x \div 2y) \div 1$	x	y	$(x \div 2y) \div 1$
0	0	$(0 \div 0) \div 1 = 0$	4	0	$4 \div 1 =$
1	0	$(1 \div 0) \div 1 = 0$	4	1	$(4 \div 2) \div 1 = 1$
2	0	$(2 \div 0) \div 1 = 1$	4	2	$(4 \div 4) \div 1 = 0$
2	1	$(2 \div 2) \div 1 = 0$	5	0	$5 \div 1 =$
3	0	$(3 \div 0) \div 1 = 2$	5	1	$(5 \div 2) \div 1 = 2$
3	1	$(3 \div 2) \div 1 = 0$	5	2	$(5 \div 4) \div 1 = 0$

31

Veamos situaciones mas complejas

Ejercicio: definir la fc. $f : \text{Nat} \rightarrow \text{Nat}$ especificada mediante $f(a) =_{\text{def}} \lfloor a/2 \rfloor$ usando sólo fcs. recursivas primitivas.

Ejercicio: definir la fc. $f : \text{Nat} \times \text{Nat} \rightarrow \text{Nat}$ donde $f(a,b) =_{\text{def}} \lfloor a/b \rfloor$ usando minimización no acotada.

¿Puede definirse esta fc. usando sólo fcs. recursivas primitivas? ¿Y si considera $f(a,0)=0$?

32

Aplicaciones

- Las fcs. recursivas han jugado un rol importante como modelo teórico de la computación.
- Diferencia con modelos anteriores:
 - uso de recursividad y composición de fcs. como herramientas básicas de resolución de problemas.
 - descripción declarativa de los problemas
 - igualdad entre dato y programa.
 - Aspectos prácticos: lenguajes funcionales (ej. LISP ó ML).

33

Síntesis

- Funciones Recursivas Primitivas
- Propiedades. Derivación formal. Predicado Recursivo Primitivo.
- Función definida por casos. Relaciones entre funciones recursivas primitivas, funciones totales y función de Ackermann.
- Funciones Recursivas Parciales: minimización no acotada.

34